播放控制

包括 打开文件, 播放, 暂停, 进度显示和调整, 播放时间显示 , 跳转

首先完成底层封装, 然后再考虑界面

在VideoPlayer类里面添加成员

```
//////////////////////////
///播放控制
void play(); //恢复播放更合适
void pause();
void stop( bool isWait);
void setFileName(const QString &fileName);
PlayerState m_playerState;
```

为了完成播放的控制, 需要在VideoPlayer中添加成员PlayerState m_playerState;播放状态是一个枚举,代表着播放状态.

```
enum PlayerState
{
    Playing = 0,
    Pause,
    Stop
};
```

VideoState 中添加播放控制

```
//////////////////////////////////////////////////
    /// 播放控制的变量
    bool isPause;//暂停标志
    bool quit; //停止
    bool readFinished; //读线程文件读取完毕
    bool readThreadFinished; //读取线程是否结束
    bool videoThreadFinished; // 视频线程是否结束
    //////////////////////////////////////////////////
    //// 跳转相关的变量
    int          seek_req; //跳转标志 -- 读线程
    int64_t       seek_pos; //跳转的位置 -- 微秒
    int          seek_flag_audio;//跳转标志 -- 用于音频线程中
    int          seek_flag_video;//跳转标志 -- 用于视频线程中
    double        seek_time; //跳转的时间(秒)  值和seek_pos是一样的
    //////////////////////////////////////////////////
```

设置文件名并播放

```
void VideoPlayer::setFileName(const QString &fileName)
{
    if( m_playerState != PlayerState::Stop  )   return;
    m_fileName = fileName;
    m_playerState = PlayerState::Playing;
    this->start();
}
```

视频的暂停和恢复播放

暂停需要一个状态,在结构体VideoState中 isPause来表示, 用这个变量,来控制三个线程(读文件线程run() , 音频解码线程, 视频解码线程)的暂停, 方法很简单就是在循环的位置, 添加判断

```
if( m_videoState.isPause )
{
        SDL_Delay(10);
        continue;
}
```

比如:

run线程中

```
while(1)
{
    if( m_videoState.quit ) break;
    if( m_videoState.isPause == true )
    {
        SDL_Delay(10);
        continue;
    }
```

音频回调函数解码函数中:

```
for(;;)
{
    if( is->quit ) return -1;

    if( is->isPause == true )
    {
        SDL_Delay(10);
        continue;
    }
```

视频线程中

```
if( is->quit ) break;
if( is->isPause )
{
    SDL_Delay(10);
    continue;
}
```

暂停

```
//暂停
void VideoPlayer::pause()
{
    m_videoState.isPause = true;
    if( m_playerState != Playing ) return;
    m_playerState = Pause;
}
```

然后添加VideoPlayer的函数回复播放play()的实现.

恢复播放

```
void VideoPlayer::play()
{
    m_videoState.isPause = false;
    if( m_playerState != Pause) return;
    m_playerState = Playing;
}
```

这里的播放是暂停到播放, 并不是停止到播放, 该项目中不会写播放列表

暂停播放时, 需要在回调函数中加入清空缓冲区，为了避免一直播放上一帧声音

```
memset( stream , 0 , len);
if(is->isPause ) return;
```

```
//回调函数中将从队列中取数据, 解码后填充到播放缓冲区.
void audio_callback(void *userdata, Uint8 *stream, int len)
{
//    AVCodecContext *pcodec_ctx    = (AVCodecContext *) userdata;
    VideoState * is = (VideoState *) userdata;

    int len1, audio_data_size;

    memset( stream , 0 , len);
    if( is->isPause ) return;
//    static uint8_t audio_buf[(AVCODEC_MAX_AUDIO_FRAME_SIZE * 3) / 2];
```

播放停止

首先, 需要分别在读取线程, 视频线程, 音频线程添加停止的相关代码

读取线程run()函数修改
读取数据包时,发现返回<0, 并不需要马上退出, 可以做一下延迟

```
if (av_read_frame(pFormatCtx, packet) < 0)
{
    DelayCount++;
    if( DelayCount>= 300)
    {
        m_videoState.readFinished = true;
        DelayCount = 0 ;
    }

    if( m_videoState.quit) break; //解码线程执行完 退出

    SDL_Delay(10);
    continue;
}
DelayCount = 0;
```

```
当退出循环时,等待音频和视频解码结束
while( !m_videoState.quit)
{
   SDL_Delay(100);
}
//回收空间
if( m_videoState.videoStream != -1)
            packet_queue_flush( m_videoState.videoq);//队列回收
if( m_videoState.audioStream != -1)
            packet_queue_flush( m_videoState.audioq); //队列回收
while( m_videoState.videoStream != -1 && !m_videoState.videoThreadFinished )
{
   SDL_Delay(10);
}
if( audioStream != -1){//回收空间
avcodec_close(pAudioCodecCtx);
   }
if( videoStream != -1 ) {//9.回收数据
      avcodec_close(pCodecCtx);
   }
avformat_close_input(&pFormatCtx);
//回收资源之后,在最后添加读取文件线程退出标志.
m_videoState.readThreadFinished = true;
m_playerState = PlayerState::Stop; //视频自动结束 置标志位
```

其中, 队列回收函数

```
void packet_queue_flush(PacketQueue *q)
{
    AVPacketList *pkt, *pkt1;
    SDL_LockMutex(q->mutex);
    for(pkt = q->first_pkt; pkt != NULL; pkt = pkt1)
    {
        pkt1 = pkt->next;
        av_free_packet(&pkt->pkt);
        av_freep(&pkt);
    }
    q->last_pkt = NULL;
    q->first_pkt = NULL;
    q->nb_packets = 0;
    q->size = 0;
    SDL_UnlockMutex(q->mutex);
}
```

视频解码线程修改

视频解码线程修改:
从队列中读取数据需要改成

```
//从队列中读取数据需要改成
//首先不能阻塞, 不然无法退出
//有读取慢, 解析快的情况(播放网络) 队列里没东西不能退出
// 音频没有解析完也不能退出
if (packet_queue_get(is->videoq, packet, 0) <= 0)
{
//播放到结束
   if( is->readFinished && is->audioq->nb_packets == 0)
   {//读线程完毕
      break;
   }else
   {
      SDL_Delay(1);  //只是队列里面暂时没有数据而已
      continue;
   }
}
```

```
if( is->quit ) break;
if( is->isPause )
{
    SDL_Delay(10);
    continue;
}
if (packet_queue_get(is->videoq, packet, 0) <= 0)
{
    if( is->readFinished && is->audioq->nb_packets == 0)
    {//读线程完毕
        break;
    }else
    {
        SDL_Delay(1);    //只是队列里面暂时没有数据而已
        continue;
    }
}
```

在while(1)循环的流程里,添加quit标志位退出

```
while(1)
{
    if( is->quit) break;
    if( is ->audioq->size == 0   ) break;
    audio_pts = is->audio_clock;
    video_pts = is->video_clock; //跳转的时候可能会用上
    if (video_pts <= audio_pts) break;
    SDL_Delay(5);
}
//在退出函数之前,设置标志位 --> 控制 读取文件线程 音频线程的结束
if( !is->quit)
{
    is->quit = true;
}
av_free(pFrame);
av_free(pFrameRGB);
av_free(out_buffer_rgb);
is->videoThreadFinished = true;

// 清屏
   QImage img; //把图像复制一份 传递给界面显示
   img.fill(Qt::black);
   is->m_player->SendGetOneImage(img); //调用激发信号的函数
```

音频解码线程修改

音频解码函数的停止:在从队列中读取时,添加退出标志

```
if(packet_queue_get(audioq, &pkt, 0) <= 0) //一定注意
{
    if( is->readFinished && is->audioq->nb_packets == 0 )
     is->quit = true;
    return -1;
}
```

构造里面的修改

```
m_playerState = PlayerState::Stop;
m_videoState.readThreadFinished = true;
```

停止函数—用于外面调用

```
//播放停止
void VideoPlayer::stop(bool isWait)//阻塞标志
{
        m_videoState .quit = 1;
        if( isWait ) //阻塞标志
        {
            while(!m_videoState.readThreadFinished )//等待读取线程退出
            {
                SDL_Delay(10);
            }
        }
        m_playerState = PlayerState::Stop;
        Q_EMIT SIG_PlayerStateChanged(PlayerState::Stop);
}
    //关闭SDL 音频设备 , 在读取文件线程中添加, 停止音频回调函数
        if (m_videoState.audioID != 0) {
        SDL_LockAudio();
        SDL_PauseAudioDevice(m_videoState.audioID,1);//停止播放,即停止音频回调函数
                SDL_CloseAudioDevice( m_videoState.audioID );
        SDL_UnlockAudio();
        m_videoState.audioID = 0;
        }
```

视频进度

```
//获取当前时间
double VideoPlayer::getCurrentTime()
{
    return m_videoState.audio_clock;
}
//获取总时间
int64_t VideoPlayer::getTotalTime()
{
    if( m_videoState.pFormatCtx )
    return m_videoState.pFormatCtx->duration;
    return -1;
}
```

```
PlayerState VideoPlayer::getPlayerState() const
{
    return m_playerState;
}
```

他们的单位都是微秒

然后在VideoPlayer中定义信号 void SIG_TotalTime(qint64 uSec); 读取文件进程函数run()函数中, 一旦开始解析视频文件, 发送此信号

```
Q_EMIT SIG_TotalTime( getTotalTime() );
```

跳转

和暂停一样，我们使用一个变量记录下是否需要跳转，在读取视频文件的线程中判断此变量，当需要跳转的时候就执行跳转操作。

跳转操作可以直接使用FFMPEG的跳转函数av_seek_frame来实现，函数原型如下：

```
int av_seek_frame(AVFormatContext *s, int stream_index, int64_t timestamp,int flags);
```

这里需要注意三个问题

跳转的实现 – 问题1

我们有2个队列，音频队列和视频队列，这2个队列里面的数据是可以播放几秒钟的，因此每次执行跳转的时候需要同时将队列清空，所以先添加一个清空队列的函数：

```
void packet_queue_flush(PacketQueue *q);
```

然后每次调用seek的时候便执行一次清空操作：

```
if (av_seek_frame(is->ic, stream_index, 0,is->seek_flags) < 0)
{    fprintf(stderr, "%s: error while seeking",is->ic->filename);
}
else {
   if (is->audioStream >= 0) {
      packet_queue_flush(&is->audioq);
   }
   if (is->videoStream >= 0) {
      packet_queue_flush(&is->videoq);
   }
}
```

跳转的实现—问题2

按照上面的方式处理,每次跳转的时候都会出现花屏的现象。
是因为解码器中保留了上一帧视频的信息，而现在视频发生了跳转，
从而使得解码器中保留的信息会对解码当前帧产生影响。
因此，清空队列的时候我们也要同时清空解码器的数据(包括音频解码器和视频解码器)，
可以往队列中放入一个特殊的packet，当解码线程取到这个packet的时候，就执行清除解码器的数据

跳转的实现—问题3

第一篇文章我们就说过视频是每隔几秒才出现一个关键帧，而av_seek_frame只能跳转到关键帧,因此这里就会存在误差。举个例子：

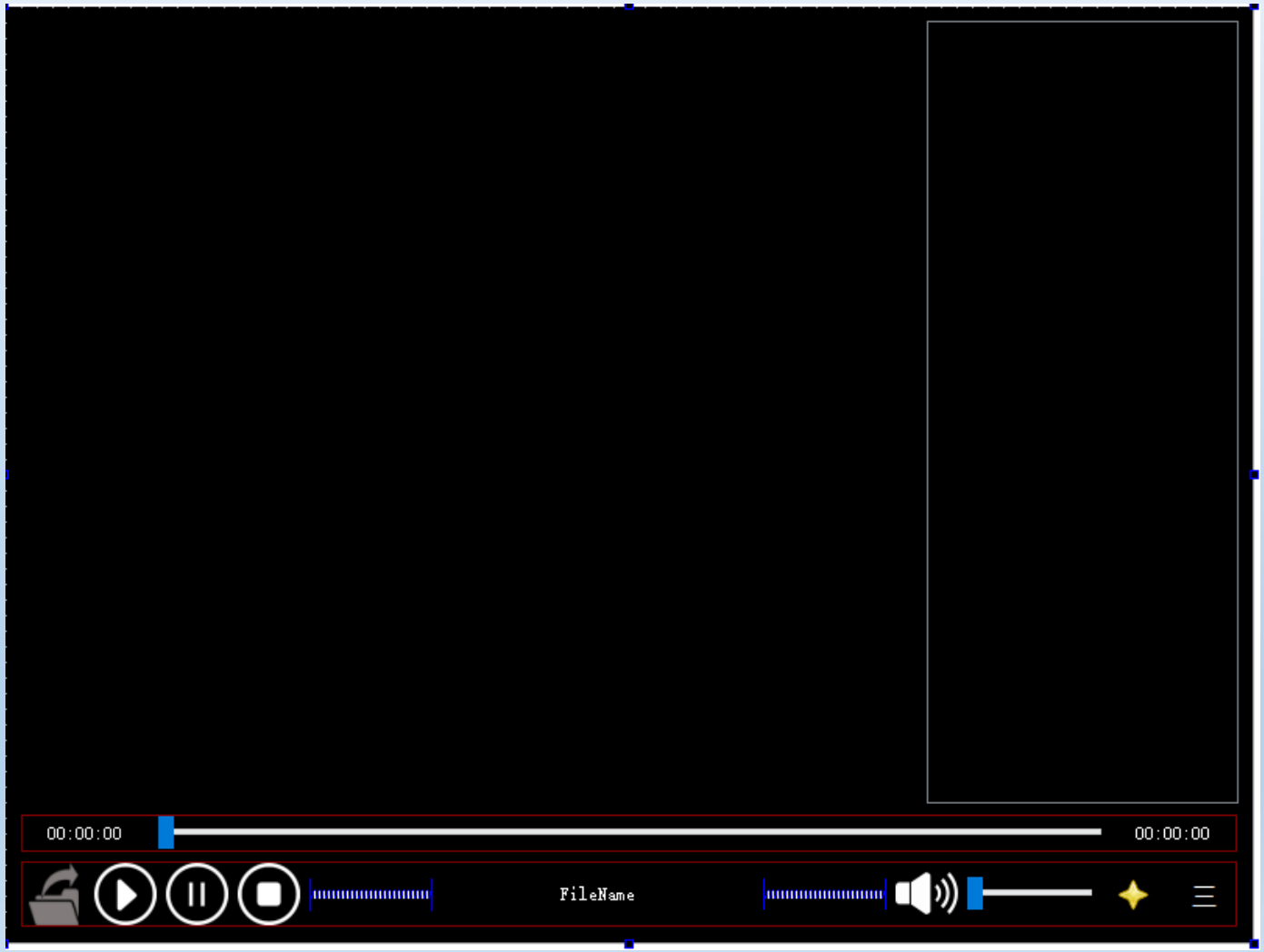
假如 我们的视频 只有在10秒和15秒上才有关键帧，这时候我们希望跳转到13秒，调用

av_seek_frame后视频只能跳到10秒或者15秒，因为解码器解码视频必须要有关键帧的信息才能解码，
因此av_seek_frame这样设定是合理的，所以我们seek之后还需要执行一些操作来处理这个情况。
那么实现就是将跳转转换到时间, 然后解码之后数据如果时间不到就舍弃掉, 不发送图片, 不播放声音.

播放控制的界面部分

界面中添加控件

底层完成了，界面多数共能通过调用底层函数即可实现



对象	类
VideoShow	QWidget
wdg_ctrl	QWidget
horizontalLayout	QHBoxLayout
lb_currentTime	QLabel
lb_totalTime	QLabel
silider_videoProcess	QSlider
horizontalLayout_2	QHBoxLayout
horizontalSlider_2	QSlider
horizontalSpacer	Spacer
horizontalSpacer_2	Spacer
lb_videoName	QLabel
pb_online	QPushButton
pb_open	QPushButton
pb_pause	QPushButton
pb_play	QPushButton
pb_showList	QPushButton
pb_stop	QPushButton
pb_volumn	QPushButton
widget	QWidget
lw_videoList	QListWidget
wdg_show	VideoItem

播放控制 采用状态机制管理

播放的状态 通过Playing控制
暂停的状态 通过Stop控制
控制状态和UI显示

播放进度采用定时器控制,定时更新时间和界面

添加成员
public:
int isStop;//代表当前是否是停止状态
QTimer *m_Timer;

```
//播放状态切换槽函数
void VideoShow::slot_PlayerStateChanged(int state){
    switch( state ){
        case PlayerState::Stop:
            m_Timer->stop();
            ui->slider_videoProcess->setValue(0);
            ui->lb_totalTime->setText("00:00:00");
            ui->lb_currentTime->setText("00:00:00");
            ui->lb_videoName->setText("");
            ui->pb_play->show();
            ui->pb_pause->hide();
            {
                QImage img;
                img.fill( Qt::black);
                ui->wdg_video->slot_setImage( img );
            }
            this->update();
            isStop = true;
        break;
        case PlayerState::Playing:
            ui->pb_play->hide();
            ui->pb_pause->show();
            m_Timer->start();
            this->update();
            isStop = false;
        break;
    }
```

播放 – 开始播放(从停止状态到播放状态) 打开文件

```
void VideoShow::on_pb_open_clicked() //打开文件
{
   //打开文件 弹出对话框 参数:父窗口, 标题, 默认路径, 筛选器
   QString path = QFileDialog::getOpenFileName(this,"选择要播放的文件" , "F:/",
         "视频文件 (*.flv *.rmvb *.avi *.MP4 *.mkv);; 所有文件(*.*);;");
   if(!path.isEmpty())
   {
      qDebug()<< path ;
      QFileInfo info(path);
      if( info.exists() )
      {
         m_player->stop( true ); //如果播放 你要先关闭
         m_player->setFileName(path);
         ui->lb_videoName->setText( info.baseName() );
         slot_PlayerStateChanged(PlayerState::Playing);
      }
      else
      {
         QMessageBox::information( this, "提示" , "打开文件失败");
      }
   }
}
```

暂停与恢复播放

```
//暂停
void VideoShow::on_pb_pause_clicked()
{
   if( isStop ) return;//停止状态自然不能播放
   m_player->pause();

   if( m_player->getPlayerState() == PlayerState::Pause)
   {
      ui->pb_play->show();
      ui->pb_pause->hide();
      this->update();
   }
}
```

```
//播放
void VideoShow::on_pb_play_clicked()
{
    if( isStop ) return;
    m_player->play();
    if(m_player->getPlayerState() ==
PlayerState::Playing)
    {
        ui->pb_play->hide();
        ui->pb_pause->show();
        this->update();
    }
}
```

播放停止

ui界面类调整

```
//停止
void VideoShow::on_pb_stop_clicked()
{
    m_player->stop(true);
}
```

获取播放全部时长

然后在VideoPlayer中定义信号 void SIG_TotalTime(qint64 uSec); 读取文件进程函数run()函数中,
一旦开始解析视频文件, 发送此信号

```
Q_EMIT SIG_TotalTime( getTotalTime() );
```

然后在ui显示界面类中, 添加

```
connect( m_player, SIGNAL( SIG_TotalTime(qint64))  ,
        this ,SLOT( slot_getTotalTime(qint64)) );
//获取视频全部时间
void VideoShow::slot_getTotalTime(qint64 uSec)
{
    qint64 Sec = uSec/1000000;

    ui->slider_videoProcess->setRange(0,Sec);//精确到秒

    QString hStr = QString("00%1").arg(Sec/3600);
    QString mStr = QString("00%1").arg(Sec/60);
    QString sStr = QString("00%1").arg(Sec%60);

    QString str = QString("%1:%2:%3").arg(hStr.right(2)).arg(mStr.right(2)).arg(sStr.right(2));
    ui->lb_totalTime->setText(str);
}
```

进度显示和控制

申请定时器成员

```
QTimer *m_Timer;
m_Timer = new QTimer; //定时器-获取当前视频时间
connect(m_Timer,SIGNAL(timeout()),this,SLOT(slot_TimerTimeOut()));
m_Timer->setInterval(500);
```

添加定时器超时的响应函数void slot_TimerTimeOut();

```
//获取当前视频时间定时器
void VideoShow::slot_TimerTimeOut()
{
    if (QObject::sender() == m_Timer)
    {
        qint64 Sec = m_Player->getCurrentTime()/1000000;
        ui->slider_videoProcess->setValue(Sec);

        QString hStr = QString("00%1").arg(Sec/3600);
        QString mStr = QString("00%1").arg(Sec/60%60);
        QString sStr = QString("00%1").arg(Sec%60);
        QString str = QString("%1:%2:%3").arg(hStr.right(2)).arg(mStr.right(2)).arg(sStr.right(2));
        ui->lb_currentTime->setText(str);
        if(ui->slider_videoProcess->value() == ui->slider_videoProcess->maximum()
                && m_Player->m_playerState == PlayerState::Stop)
        {
            slot_PlayerStateChanged( PlayerState::Stop );
```

UI调整播放进度

自定义重写QSlider类型.

```
class VideoSlider : public QSlider
{
    Q_OBJECT
public:
    explicit VideoSlider(QWidget *parent = 0);
    ~VideoSlider();
signals:
    void SIG_valueChanged(int);
protected:
    void mousePressEvent(QMouseEvent *event);
};
```

只需要重写鼠标点击事件.

```
void VideoSlider::mousePressEvent(QMouseEvent *event)
{
int value = QStyle::sliderValueFromPosition(
minimum(), maximum(), event->pos().x(), width());
    setValue(value);
    emit SIG_valueChanged(value);
}
```

将ui->slider_videoProcess提升为VideoSlider.

跳转函数

添加调整进度的信号槽连接

```
connect( ui->slider_videoProcess , SIGNAL(SIG_valueChanged(int)) ,
this, SLOT(slot_videoSliderValueChanged(int)) );
```

```
//视频进度通过手动点击改变
void VideoShow::slot_videoSliderValueChanged(int value)
{
    if( QObject::sender() == ui->slider_videoProcess )
    {
        m_Player->seek((qint64)value*1000000);  //value 秒
    }
}
```